

Python aplicado al análisis y procesamiento de datos

Facilitador: Javier Sánchez Padilla.

Repaso general sobre el lenguaje

Python es un lenguaje de programación poderoso y versátil que ha ganado popularidad debido a su facilidad de uso, legibilidad y amplia gama de aplicaciones.

PEP 8

¿Qué es?

- La guía de estilo PEP 8 es un conjunto de recomendaciones para escribir código Python. El objetivo es que el código sea más legible y fácil de leer.

Ejemplos de reglas PEP 8:

- Longitud máxima de líneas
- Espacios en blanco
- Nombres de variables y funciones

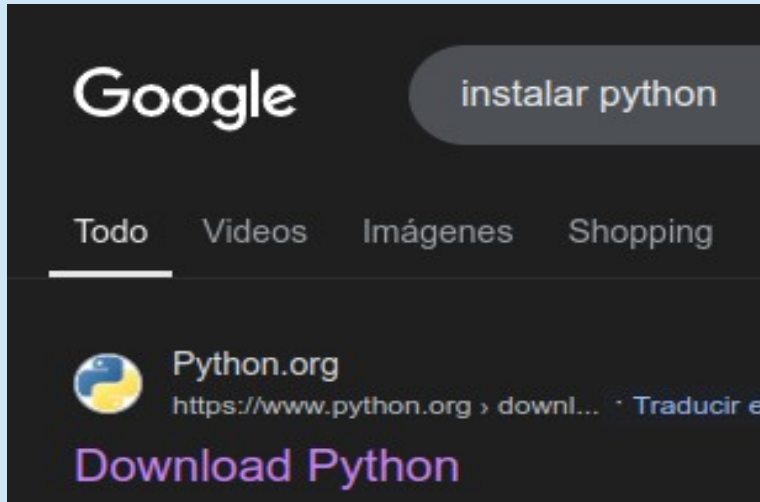
PEP 8

Documento oficial.

- <https://peps.python.org/pep-0008/>
- En Google podemos buscar “PEP 0008”
- Aplicación de guías en línea para PEP8.
- En Google buscar “python formatter on line”
- Ej. <https://codebeautify.org/python-formatter-beautifier>

Instalar Python

- Dirección oficial <https://www.python.org/downloads/>
- En Google podemos buscar. “Instalar python.”



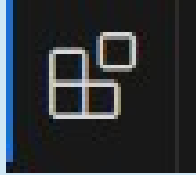
Instalar un editor de código.

Descargar Visual Studio Code desde:

- <https://code.visualstudio.com/>
- En Google buscar “visual studio code”

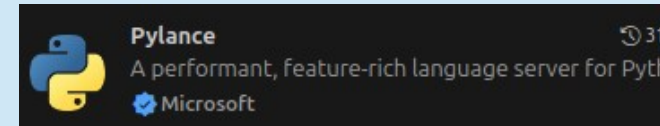
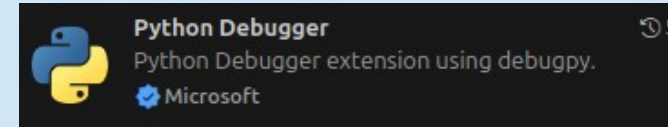
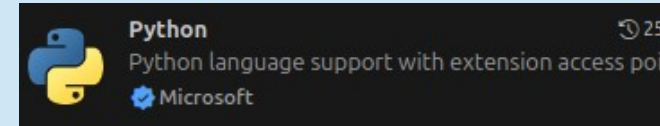
En caso de querer personalizar puede desde archivo → preferencias → Tema

Instalar las extensiones necesarias.



Algunas de las características de la extensión Python son:

- IntelliSense (ayuda a completar código).
- Linting (analizador de código).
- Depuración
- Navegación por el código
- Formateado del código
- Refactorización (modif. Código sin cambio en comportamiento)
- Explorador de variables
- Explorador de pruebas



Creando un proyecto desde Visual Studio Code.

- Creación de la carpeta de trabajo.
- Abrir la carpeta desde visual studio Code.
- Crear un archivo con extensión .py

Print() ¿qué podemos imprimir con print()?

- Valores numéricos.
- Operaciones matemáticas.
- Cadenas de texto
- Unir cadenas (mediante `,`), así como cadenas y otros tipos de datos.
- Concatenar cadenas (operador `+`).
- Textos con operaciones matemáticas

Print()

- Uso de tabulaciones (\t)
- Uso de <Enter> salto de linea (\n)
- Cuando unimos valores mediante coma (,) podemos definir un separador (sep=)
- Definir el final de la linea (end=)
- Hacer uso de f-string

Print() Formateo de la salida

Existe un patron en los f-string dentro de las llaves, dos puntos (:)

Mini-lenguaje de especificación de formato de Python.

La estructura completa es:

{variable : [relleno][alineación][ancho][separador][.precisión][tipo]}

(No necesitas aprenderte todos de memoria. Los más usados son)

serán {variable: **.2f** (dinero/ciencias) ,**.2f** (contabilidad) y **%d/%m/%Y** (fechas)

Print() Formateo de la salida

{variable : [relleno][alineación][ancho][separador][.precisión][tipo]}

- Dos decimales {a:.2f}
- Dos decimales con separador de miles {a:,.2f}

- **Formatos de fecha**

%d Día del mes (01 a 31)
%Y Año completo (2026)
%A Nombre del día (Lunes)

%m Mes en número (01 a 12)
%H:%M Horas y minutos.
%B Nombre del mes (Febrero)

Print() Formateo de la salida

{variable : [relleno][alineación][ancho][separador][.precisión][tipo]}

- Formateo de salida de texto:
 - < Alinea a la izquierda (por defecto para texto).
 - > Alinea a la derecha (por defecto para números).
 - ^ Centra el contenido.
- f"|{texto:>10}|" {texto:=^20}

Print() Formateo de la salida

{variable : [relleno][alineación][ancho][separador][.precisión][tipo]}

- Sistemas numéricos

Para convertir números a otras bases, es muy fácil:

b Binario

x Hexadecimal.

o Octal.

{numero:b} {numero:x} {numero:o}

¿Para que sirven las variables?

¿Para qué sirven las variables numéricas?

- Almacenar datos: Puedes guardar cualquier número, ya sea un entero (1, 2, 3, ...) o un decimal (1.5, 3.14, ...).
- Realizar cálculos: Puedes hacer operaciones matemáticas con las variables numéricas, como sumas, restas, multiplicaciones y divisiones.
- Comparar valores: Puedes verificar si dos variables numéricas son iguales, si una es mayor que otra, etc.

Variables en PYthon

- Creación y Asignación de Variables
- Tipado Dinámico y fuertemente tipado (previene operaciones indebidas)
- Reglas y Convenciones de Nombres (PEP 8)
 - Sensible a mayúsculas y minúsculas
 - Caracteres permitidos (A-Z, a-z, 0-9 y _)
 - Inicio del nombre
 - Palabras reservadas

Variables en Python

- Convenciones recomendadas por PEP 8 (snake case)
- Asignación Múltiple y Reorganización de Valores (`a, b = 1, 2`)
- Asignar un mismo valor a varias variables (`a, b = 1`)
- Intercambiar valores entre variables (Swap) (`a, b = b, a`)
- Las Variables en Python son Referencias a Objetos

Tipos de datos básicos en Python.

Tipos de Datos Básicos

- Numéricos:
 - **int**: Números enteros, como 2, -5, 0.
 - **float**: Números de punto flotante (con decimales), como 3.14, -2.5.
- Texto:
 - **str**: Cadenas de caracteres, como "Hola", 'Python'(.upper(), lower(), capitalize(), count(arg), replace(str_orig, str_reemplazo)).
- Booleanos.
 - **bool**: Valores lógicos, True o False.

Función TYPE()

La función **type()** en Python es una herramienta fundamental para determinar el tipo de dato de un objeto.

¿Qué hace exactamente?

- Toma un objeto como argumento: Se pueden pasar cualquier variable, valor o incluso el resultado de una expresión.
- Devuelve el tipo del objeto: Te indica a qué clase pertenece el objeto.

Para que sirve la función TYPE()

- Depuración: Si estás obteniendo resultados inesperados, puedes usar **type()** para verificar si estás trabajando con el tipo de dato correcto.
- Control de flujo: Puedes crear condicionales basados en el tipo de dato (int, str, bool).
- Conversiones de tipo: Puedes usar **type()** para saber qué tipo de conversión es necesaria.

Programa 01. Con los siguientes elementos crear un recibo de compras

```
from datetime import datetime

# Datos de la compra
cliente = "Juan Pérez"
fecha = datetime.now()
producto1 = "Laptop Gamer"
precio1 = 25400.50
producto2 = "Mouse"
precio2 = 550.00
impuesto_porcentaje = 0.16

subtotal = precio1 + precio2
iva = subtotal * impuesto_porcentaje
total = subtotal + iva
```

```
-----
                        MI TIENDITA TECH
-----
Fecha: 16/08/2026 12:48
Cliente: Juan Pérez
-----
Producto                                Precio
Laptop Gamer                           $    25,400.50
Mouse                                   $      550.00
-----
Subtotal:                              $    25,950.50
IVA (16%):                              $      4,152.08
TOTAL:                                  $    30,102.58
-----
                        ¡GRACIAS POR SU COMPRA!
```

Operadores matemáticos.

Operador	Descripción	Ejemplo
+	Suma	<code>x + y</code>
-	Resta	<code>x - y</code>
*	Multiplicación	<code>x * y</code>
/	División (resultado en punto flotante)	<code>x / y</code>
//	División entera (resultado redondeado hacia abajo)	<code>x // y</code>
%	Módulo (resto de la división)	<code>x % y</code>
**	Exponenciación	<code>x ** y</code> (x elevado a la y)

Operadores compuestos

Operador	Nombre	Ejemplo	Equivale a
+=	Suma y asigna	$x += 2$	$x = x + 2$
-=	Resta y asigna	$x -= 2$	$x = x - 2$
*=	Multiplica y asigna	$x *= 2$	$x = x * 2$
/=	División y asigna	$x /= 2$	$x = x / 2$
**=	Potencia y asigna	$x **= 2$	$x = x ** 2$
//=	División entera y asigna	$x //= 2$	$x = x // 2$
%=	Residuo (módulo) y asigna	$x \% = 2$	$x = x \% 2$

Programa 02. Uso de los operadores

Crear un código que simule el funcionamiento de una máquina de cambio de divisas. (dólares - pesos)

La máquina va a necesitar disponer de variables que le brinden la siguiente información:

Nombre del usuario, Fecha en que se realiza la operación, Momento del día (día, tarde o noche), Cantidad de dólares a cambiar.

Con todo eso, la máquina va a imprimir en pantalla (en diferentes líneas por supuesto) un mensaje que incluya los siguientes Requisitos:

- Un saludo de bienvenida
- Información de la cantidad de dólares para el cambio
- Información del tipo de cambio en pesos que va a recibir
- Detalle específico de cuántos billetes de 200 pesos, 100 pesos, 50 pesos, 20 pesos, monedas de 10, 5 y 1 peso, y el saldo en monedas que le serán entregados
- Un saludo de despedida

Índexar un string en Python.

Índexar un string en Python es como asignar un número a cada letra o carácter dentro de ese string, lo que te permite acceder a elementos específicos de forma individual.

¿Cómo funciona?

- El primer carácter siempre tiene índice 0: Esto significa que el primer carácter del string se encuentra en la posición 0.
- Los índices aumentan de izquierda a derecha: El segundo carácter tiene índice 1, el tercero tiene índice 2, y así sucesivamente.
- Se pueden usar índices negativos: Los índices negativos cuentan desde el final del string. Por ejemplo, -1 se refiere al último carácter, -2 al penúltimo, etc.

¿Para qué sirve?

- Acceder a caracteres específicos: Puedes extraer una letra en particular para realizar operaciones con ella.
- Recorrer un string: Puedes iterar sobre cada carácter del string para realizar algún procesamiento.
- Buscar subcadenas: Puedes buscar si una determinada subcadena se encuentra dentro del string.
- Modificar strings: Aunque los strings son inmutables (no se pueden modificar directamente), puedes crear nuevos strings basados en partes de otros.

Como aplicarlo

```
palabra = "Hola a todos"
print(palabra)
print(palabra[0])
print(palabra[1])
print(palabra[2])
print("\nAcceder mediante valores negativos")
print(palabra[-1])
print(palabra[-2])
print(palabra[-3])
print(palabra[-4])
print("\nObteniendo segmentos de cadena")
print(palabra[0:4]) # El último valor no es incluyente
print(palabra[:4]) # El último valor no es incluyente
print(palabra[:40]) # No importa que nos pasemos
# imprimir desde el inicio hasta el final saltando de dos en dos
print(palabra[0:10:2])
print(palabra[0:10:2].upper())
```


Programa 03. Slicing

Vas a crear un programa que le pida al usuario que ingrese un texto de al menos 10 palabras. Tu programa va a procesar ese texto, lo va a procesar y le va a devolver un análisis detallado, que incluya:

- Contar el número total de caracteres en el texto.
- Contar el número de caracteres sin incluir los espacios.
- Contar la cantidad de vocales que hay en el texto
- Contar el número total de palabras en el texto ingresado
- Reemplazar todos los espacios por guiones medios (-)
- Cambia las mayúsculas a minúsculas y las minúsculas a mayúsculas
- Eliminar la primer palabra del texto

Listas.

Una lista en Python es una colección ordenada de elementos. Estos elementos pueden ser de cualquier tipo: números, palabras (cadenas de texto), otros tipos de datos, e incluso otras listas.

Características principales:

- Ordenadas: Los elementos de una lista tienen un orden específico, el primero, el segundo, el tercero, etc.
- Mutables: Puedes cambiar los elementos de una lista, agregar nuevos o eliminar algunos.
- Índices: Cada elemento de una lista tiene un índice numérico, que comienza en 0. Esto te permite acceder a elementos específicos de la lista.

Listas.

Pensemos que las listas son como una caja donde puedes guardar diferentes objetos. En Python, esa caja es una lista. Los objetos que guardas dentro pueden ser números, palabras (texto), o incluso otras listas. Lo interesante de estas cajas es que puedes agregar más objetos, quitar algunos, o cambiar el orden de los que ya tienes.

¿Para qué sirven las listas?.

Almacenar datos relacionados: Por ejemplo, una lista de nombres, una lista de calificaciones, una lista de coordenadas.

- Organizar información: Puedes agrupar datos de diferentes tipos en una sola lista.
- Iterar sobre elementos: Puedes recorrer cada elemento de una lista y realizar operaciones con ellos.

¿Qué más puedo hacer con las listas?

- **Acceder a elementos:** Puedes seleccionar un elemento específico de la lista usando su índice (la posición que ocupa, empezando desde 0). Por ejemplo, `mi_lista[0]` te dará el primer elemento (1 en este caso).
- **Agregar elementos:** Puedes agregar elementos al final de la lista con el método `append()`. Por ejemplo, `mi_lista.append("adios")` agregará la palabra "adios" al final de la lista.
- **Eliminar elementos:** Puedes eliminar elementos usando el método `remove()`. Por ejemplo, `mi_lista.remove("hola")` eliminará la palabra "hola" de la lista.
- **Cambiar elementos:** Puedes cambiar el valor de un elemento usando su índice. Por ejemplo, `mi_lista[1] = 4` cambiará el segundo elemento de la lista (que era 2) por el número 4.

Programa 04

- Crea una lista que contenga al menos cinco nombres de ciudades. Luego, escribe un programa que imprima (por separado) el nombre de la primera y la última ciudad almacenadas en tu lista. Puedes usar esta lista como ejemplo, y agregar o quitar ciudades para ir probando con listas de diferentes longitudes

Programa 05

En el siguiente ejercicio por cada modificación imprimir su resultado en pantalla.

- Crea una lista llamada números con cinco elementos: 1, 2, 3, 4, y 5.
- Añadir un elemento al final: Se utiliza el método adecuado para añadir el número 6.
- Eliminar el primer elemento:
- Insertar un elemento en una posición específica: Se usa el método adecuado para insertar el número 7 en la posición con índice 1 de la lista.
- Ordenar la lista:

Programa 06

- Crea dos listas: lista1 con los elementos 1, 2, 3, y lista2 con los elementos 4, 5, 6.
- Concatena lista1 y lista2 para formar una nueva lista llamada combinada.
- Multiplica el contenido de la lista lista1 por 3 para crear una nueva lista llamada repetida. Esto hará que los elementos de lista1 se repitan tres veces en repetida.
- Creación de Sublistas: A partir de la lista combinada creada en el paso 2, crea una sublista llamada sublista que contenga solo los elementos desde el índice 1 hasta el índice 4 (sin incluir el elemento en el índice 4).

Tuplas

Una tupla en Python es una colección ordenada de elementos, al igual que una lista. Sin embargo, a diferencia de las listas, las tuplas son inmutables, lo que significa que una vez creadas, no puedes cambiar sus elementos.

Características principales:

- Ordenadas: Los elementos de una tupla tienen un orden específico.
- Inmutables: No puedes agregar, eliminar o modificar elementos después de crear la tupla.
- Índices: Cada elemento tiene un índice numérico, al igual que en las listas.

.

Tuplas

Comparemos las tuplas como una caja fuerte, donde una vez que guardas algo en ella, ya no puedes sacar nada ni añadir nada más. Las tuplas son como esa caja fuerte para tus datos.

-

Programa 07

- Dada una tupla llamada `info_ciudades` que contiene tres tuplas dentro. Cada tupla interna representa una ciudad y contiene el nombre de la ciudad, la población y el país. ("Buenos Aires", 3000000, "Argentina"), ("Madrid", 3200000, "España"), ("Tokio", 13929286, "Japón"). Imprime un mensaje para cada ciudad siguiendo el formato siguiente: "La ciudad de [nombre de la ciudad] en [país] tiene una población de [población] habitantes" Utiliza los índices requeridos para acceder a los datos específicos de cada país, crea las variables que consideres necesarias y con el empleo de cadenas literales imprime tres mensajes en el orden siguiente: "La ciudad de Buenos Aires en Argentina tiene una población de 3000000 habitantes" "La ciudad de Madrid en España tiene una población de 3200000 habitantes" "La ciudad de Tokio en Japón tiene una población de 13929286 habitantes"

Diccionarios

Imaginemos que tenemos una agenda de contactos. En esa agenda, cada persona tiene un nombre (una clave) y una información asociada (un valor), como su número de teléfono. Los diccionarios en Python funcionan de manera similar.

¿Qué es un diccionario en Python?

- Un diccionario es una colección de datos donde cada elemento se almacena como un par **clave-valor**. La clave es única y sirve para identificar el valor asociado. Por ejemplo, en nuestra agenda, el nombre sería la clave y el número de teléfono sería el valor.

¿Para qué sirven los diccionarios?

- Organizar información: Los diccionarios son ideales para almacenar datos relacionados de forma estructurada.
- Acceder a datos rápidamente: Puedes acceder a un valor utilizando su clave.
- Representar datos complejos: Los diccionarios pueden contener otros diccionarios, listas y otros tipos de datos.

Programa 08

- Dado el siguiente diccionario edades, que asocia nombres de personas con su edad: `edades = {"Juan": 28, "Elena": 32, "Marcos": 17}` Realiza las siguientes tareas: Accede a la edad de Elena e imprímela. Modifica la edad de Juan a 29 años e imprime el resultado. Añade un nuevo par clave-valor, asociando a "Luisa" con 25 años e imprime el diccionario completo actualizado.

Programa 09

Con el siguiente diccionario productos que contiene productos como claves y sus precios como valores: `productos = {"manzana": 20, "platano": 15, "cereza": 30}`

Realiza las siguientes operaciones usando métodos apropiados de diccionarios e imprime el resultado en cada operación:

- Guarda todas las claves del diccionario en una variable.
- Guarda todos los valores del diccionario en una variable
- Añade un nuevo producto llamado "naranja" con un precio de 30.
- Actualiza el precio de el "platano" a 12
- Imprime todos los pares clave-valor del diccionario actualizado.

Manejo de conjuntos

- Los conjuntos (o set en Python) son una estructura de datos nativa inspirada directamente en la teoría de conjuntos de las matemáticas.
- Se caracterizan principalmente por dos reglas fundamentales: no admiten elementos duplicados y no guardan un orden específico.

Características Principales de un Conjunto

- **Unicidad:** Si intentas agregar un valor repetido, Python simplemente lo ignora.
- **Desordenados:** Los elementos no tienen una posición fija (no existe el concepto de "índice 0" o "último elemento"). No puedes hacer `mi_set[0]`.
- **Mutables:** Puedes agregar o eliminar elementos de un conjunto.
- **Elementos Inmutables:** Aunque el conjunto en sí es mutable, todos los elementos que almacenes dentro de él deben ser de un tipo inmutable (`int`, `float`, `str`, `tuple`). No puedes meter listas ni diccionarios dentro de un conjunto.

Operaciones

- **UNION** ($\text{conj_a} \mid \text{conj_b}$ ó $\text{conj_a.union(conj_b)}$)
- **INTERSECCION** ($\text{conj_a} \& \text{conj_b}$ ó $\text{conj_a.intersection(conj_b)}$)
- **DIFERENCIA** ($\text{conj_a} - \text{conj_b}$ ó $\text{conj_a.difference(conj_b)}$)
- **DIFERENCIA SIMETRICA** ($\text{conj_a} \wedge \text{conj_b}$ ó $\text{conj_a.symmetric_difference(conj_b)}$)

Booleanos

Los booleanos en Python son como pequeños interruptores que solo pueden tener dos valores: True (verdadero) o False (falso). Son como decir "sí" o "no" en una pregunta.

¿Para qué sirven los booleanos?

- Los booleanos se utilizan para tomar decisiones en nuestros programas. Por ejemplo, imagina que quieres saber si una persona es mayor de edad. Puedes usar un booleano para almacenar esta información:

Operadores lógicos (booleanos).

Los operadores booleanos son herramientas fundamentales en Python que nos permiten combinar y manipular valores booleanos (True o False) para crear expresiones lógicas más complejas. Estos operadores son esenciales para tomar decisiones en nuestros programas y controlar el flujo de ejecución.

Principales operadores booleanos en Python:

- And Devuelve True si ambas expresiones a su izquierda y derecha son True.
- Or Devuelve True si al menos una de las expresiones a su izquierda o derecha es True.
- Not Invierte el valor booleano de la expresión. Si es True, lo convierte en False y viceversa. Es como decir "no".

Operadores relacionales.

- `==` Igual a. Devuelve True si ambos valores son iguales.
- `!=` Diferente de. Devuelve True si ambos valores son diferentes.
- `<` Menor que. Devuelve True si el valor de la izquierda es menor que el de la derecha.
- `>` Mayor que. Devuelve True si el valor de la izquierda es mayor que el de la derecha.
- `<=` Menor o igual que. Devuelve True si el valor de la izquierda es menor o igual que el de la derecha.
- `>=` Mayor o igual que. Devuelve True si el valor de la izquierda es mayor o igual que el de la derecha.

IF

if en Python es como un punto de decisión en tu programa. Imagina que estás conduciendo y llegas a un cruce. Si el semáforo está en verde (condición), avanzas (acción). Si está en rojo (condición), te detienes (acción).

En Python, el if funciona de manera similar:

- Si se cumple una determinada condición (es decir, si una expresión es verdadera), se ejecuta un bloque de código.
- Si la condición no se cumple, se puede opcionalmente ejecutar otro bloque de código (con else).

Programa 10

evaluar un valor entre 0 y 100 y determinar el rango donde se encuentre

0 - 10 en el rango de 0 a 10

11 - 20 en el rango de 11 a 20

21 - 30 en el rango de 21 a 30

....

91 - 100 en el rango de 91 a 100

Programa 11

Solicitar la edad de una persona, si $\text{edad} < 18$ años imprimir que es menor de edad, en otro caso es menor de edad

IF parte 2

If condicion:

Sentencias

Elif condicion2:

Sentencias

Elif condicion3:

Sentencias

Else:

Sentencias

RESOLVER EL PROBLEMA 11 MEDIANTE ESTA ESTRUCTURA

Programa 12

Solicitar la edad de una persona, si $\text{edad} < 18$ años imprimir que es menor de edad, en otro caso es menor de edad

Estructuras de control.

- Condicionales: Son como bifurcaciones en un camino. Te permiten tomar decisiones en función de si una condición se cumple o no. Por ejemplo, si una variable es mayor que otra, se ejecutará un bloque de código, sino, se ejecutará otro.
- Bucles: Son como ciclos repetitivos. Te permiten ejecutar un bloque de código varias veces mientras se cumpla una condición. Por ejemplo, puedes imprimir los números del 1 al 1000 utilizando un bucle.

Ciclos.

- Los ciclos, también conocidos como bucles, son una herramienta fundamental en programación que nos permite repetir un bloque de código una y otra vez hasta que se cumpla una determinada condición. Son como una especie de bucle o ciclo que se ejecuta de forma repetitiva.

¿Por qué son útiles los ciclos?

- Automatización: Evitan tener que escribir el mismo código varias veces.
- Iteración: Permiten procesar grandes cantidades de datos de forma eficiente.
- Repetición: Ejecutan un bloque de código mientras se cumpla una condición.

Tipos de ciclos en Python:

Ciclo for:

- Se utiliza cuando sabemos de antemano cuántas veces queremos que se repita el código. Es ideal para iterar sobre secuencias como listas, tuplas o cadenas de caracteres.

for elemento in secuencia:

 # Código a ejecutar para cada elemento

Programa 13

- Escribe un programa que itere sobre una lista de cadenas que representan nombres de países por ejemplo: ['Francia', 'Alemania', 'España', 'Italia', 'Portugal', 'Chile']. Dentro del bucle, usa una declaración if para imprimir el nombre del país solo si contiene la letra 'a'. Asegurate que tu lista tenga por nombre: `países` Podrás agregar o eliminar los países que desees a la lista `países` proporcionada inicialmente.

Range()

La función `range()` en Python es una herramienta muy útil para generar secuencias de números de manera sencilla y eficiente.

- Si requerimos generar una lista de números del 0 al 9. En lugar de escribir cada número manualmente, puedes usar `range()`:
- `range()` no te devuelve directamente una lista, sino un objeto de tipo `range`. Pero este objeto representa una secuencia de números que puedes iterar, por ejemplo, en un bucle `for`:
-

Range()

- `range(5)`: Genera los números del 0 al 4 (5 números en total).
- `range(2, 8)`: Genera los números del 2 al 7.
- `range(1, 10, 2)`: Genera los números impares del 1 al 9 (saltando de 2 en 2).
- `range(10, 0, -1)`: Genera los números del 10 al 1 en orden descendente.

Programa 14

- Escribe un programa que use la función `range()` para sumar todos los números del 1 al 100. Para ello sigue los siguientes pasos: Inicializa la variable `suma = 0`. Usa un bucle `for` para iterar sobre cada número del 1 al 100, incluyendo el 100. Imprime el resultado de la suma empleando el siguiente mensaje: "La suma de todos los números del 1 al 100 es: {suma}" Pista: En cada paso del bucle, actualiza la variable `suma` incrementándola con el valor del número actual en la iteración. De esta manera, `suma` acumulará el total de todos los números procesados hasta ese momento.

Tipos de ciclos en Python:

Ciclo while:

- Se utiliza cuando no sabemos exactamente cuántas veces se ejecutará el código, sino que queremos que continúe mientras se cumpla una condición.

while condición:

Código a ejecutar mientras la condición sea verdadera

Programa 15

- Utilizando un bucle while, escribe un programa que cuente desde 0 hasta 15, pero que omita los números 5 y 10. Usa la declaración continue para lograr este comportamiento, e imprime cada número en una línea nueva.

Programa 16

- Escribe un programa en Python que utilice un bucle while para imprimir números empezando desde 1. Utiliza la declaración break para terminar el bucle si se encuentra un número múltiplo de 4 (luego de haberlo impreso). Pista: Para determinar si un número es múltiplo de 4 puedes hacerlo de la siguiente forma: `if x % 4 == 0:` Donde x representa el número actual en la secuencia. Si x es múltiplo de 4, esta condición se evaluará como verdadera, y podrás utilizarla para decidir cuándo ejecutar la declaración break para salir del bucle.

Funciones:

- Las funciones son uno de los pilares más importantes de la programación en Python. En términos sencillos, una función es un bloque de código ejecutable y reutilizable diseñado para realizar una tarea específica.
- El principio central detrás del uso de funciones es la regla de desarrollo DRY (Don't Repeat Yourself / No te repitas): en lugar de escribir las mismas líneas de código una y otra vez, encapsulas esa lógica dentro de una función y la ejecutas cada vez que la necesites.

Programa 17:

Sistema de Cotización de Envíos y Descuentos

Objetivo Académico

Diseñar un programa modular en Python que calcule el costo total de una compra en línea considerando:

- El cálculo de impuestos.
- El procesamiento de cupones de descuento.
- El cálculo de costo de envío según la distancia.
- La generación de un resumen formateado para el cliente.

Definición de funciones:

1. **calcular_impuesto(subtotal, porcentaje_impuesto=16)**

Propósito: Calcular el monto correspondiente a impuestos (por ejemplo, IVA).

Parámetros:

 subtotal (float): El costo base de los productos.

 porcentaje_impuesto (float, opcional): Por defecto debe ser 16.

Retorno:

 El valor numérico del impuesto calculado (float).

Definición de funciones:

2. **aplicar_cupon(subtotal, codigo_cupon)**

Propósito: Determinar el descuento aplicable según una clave.

Parámetros:

 subtotal (float).

 codigo_cupon (str).

Lógica:

 Si el cupón es "DESCUENTO10", aplica un 10% de descuento.

 Si el cupón es "SUPER20", aplica un 20% de descuento.

 Si el cupón no es válido o está vacío, el descuento es 0.

Retorno:

 El monto del descuento en dinero (float).

Definición de funciones:

3. **calcular_envio(distancia_km, envio_prioritario=False)**

Propósito: Calcular la tarifa de entrega.

Lógica:

- La tarifa base es de \$50.00 pesos.

- Se suman \$5.00 pesos adicionales por cada kilómetro recorrido.

- Si envio_prioritario es True, se le suma un recargo de \$100.00 pesos.

Retorno:

- El costo total del envío (float).

Definición de funciones:

4. **generar_ticket(nombre_cliente, subtotal, descuento, impuesto, envio)**

Propósito: Imprimir un desglose limpio y bien formateado usando f-strings.

Retorno:

No retorna nada (None), solo imprime en pantalla.

Fin de la parte uno.